

PasswordGuard: An Experimental Analysis of Password Strength and Authentication Security

Duc Nhat Nguyen
Faculty of Information Technology
Vietnam National University
Ho Chi Minh City, Vietnam
ducnhatg@gmail.com
Student Research Team

Course: English Writing and
Presentation Skills
Faculty of Information Technology
Vietnam National University
team@passwordguard.edu.vn
Academic Advisor

Department of Computer Science
Vietnam National University
Ho Chi Minh City, Vietnam
advisor@vnu.edu.vn

ABSTRACT

Password-based authentication remains the foundational access control mechanism across modern computing systems despite severe vulnerabilities stemming from human cognitive memory limits and predictable pattern selection. This paper presents PasswordGuard, an educational client-side web application designed to evaluate credential complexity, estimate theoretical search spaces, and demonstrate cryptographic hashing and salting mechanics without server-side data persistence. The system implements a transparent 0–100 heuristic scoring algorithm, theoretical character-space entropy estimation under an idealized uniform-selection assumption, and illustrative brute-force guessing projections across assumed threat models. To evaluate the functional behavior of the heuristic analyzer, an experiment was conducted on a controlled dataset of 150 synthetic passwords evenly partitioned across five structural categories ($n = 30$). The evaluation demonstrated consistent heuristic responses: 16-character pseudo-random strings (Category E) attained the highest mean score (90.0/100, 100% Strong), and multi-word passphrases (Category D) achieved the highest mean theoretical character-space entropy estimate (182.9 bits, 70.0% Strong) despite using only two character pools. Conversely, superficially complex passwords containing dictionary roots and appended symbols (Category B) were penalized heavily, averaging 10.5/100 (96.7% Weak). Because the experiment evaluates the application's internal heuristic model on synthetic samples, it does not serve as an independent real-world cracking benchmark. Ultimately, the project demonstrates that password length exponentially expands theoretical search spaces and emphasizes that effective authentication requires defense in depth, combining memorable passphrases with password managers, slow memory-hard key derivation functions such as Argon2id, rate limiting, and multi-factor authentication.

CCS Concepts

• Security and privacy ~ Authentication; Usability in security and privacy; Cryptographic protocols; Hash functions.

Keywords

password authentication; password strength; heuristic scoring; theoretical entropy; cryptographic salting

1. INTRODUCTION

Authentication is the cornerstone of information security, serving as the gateway that verifies whether an entity requesting access to a system is indeed authorized to do so [13]. Among the primary authentication factors—knowledge ("something you know"), possession ("something you have"), and inherence ("something you are")—knowledge-based passwords remain the most widely deployed mechanism worldwide due to their legacy support, minimal implementation overhead, and intuitive conceptual model [2].

However, the human factors governing password selection introduce severe systemic vulnerabilities. Studies examining leaked credentials consistently reveal that users select predictable patterns, dictionary terms, common date sequences, and simple

character substitutions (such as replacing 'a' with '@') to accommodate memorization constraints [15]. Furthermore, users routinely reuse passwords across diverse services, magnifying the blast radius of single-site breaches through automated credential stuffing attacks [14].

At the same time, computing capabilities available to threat actors have expanded by multiple orders of magnitude. Commercial graphics processing units (GPUs) and specialized Application-Specific Integrated Circuits (ASICs) enable offline cracking rigs to test fast hash candidates at large scale against captured authentication databases [12]. Consequently, intuitive human conceptions of "complexity" fail to reflect the mathematical realities of cryptographic search spaces.

1.1 Objectives of this Work

To bridge this educational gap for undergraduate students in Information Technology, this project develops **PasswordGuard**, an educational software platform and empirical evaluation framework with the following specific objectives:

- Provide a transparent, client-side password analysis system that quantifies structural properties, character diversity, and vulnerability patterns.
- Educate users on the mathematical relationship between password length (L), character pool size (R), and theoretical character-space entropy (E).
- Illustrate brute-force search space complexity under educational online vs. offline guessing rate scenarios.
- Provide an interactive demonstration of one-way cryptographic hashing (using browser-native Web Crypto SHA-256) and evaluate the necessity of cryptographic salts.
- Contrast general-purpose fast hashing algorithms with production-grade, memory-hard Key Derivation Functions (KDFs) such as Argon2id [1].
- Maintain privacy-by-design standards by operating client-side in browser memory without application-level storage or transmission of user-entered text.

2. BACKGROUND AND RELATED WORK

2.1 The Prover-Verifier Paradigm

Password authentication is fundamentally a prover-verifier protocol. A client establishes an identity identifier (username) and a secret password (S). Upon subsequent authentication attempts, the client presents S . The server does not need to store the plaintext secret; rather, it validates whether the presented password produces a digest matching the stored cryptographic verifier (salted hash) [13].

2.2 Character Spaces and Combinatorial Complexity

The mathematical difficulty of an exhaustive brute-force search against a random password depends on two parameters: alphabet size (R , the number of discrete symbols available in candidate character pools) and length (L , total characters). The total search space size N is expressed exponentially: $N = R^L$.

Because length appears as an exponent while alphabet size acts only as the base, increasing length yields exponential increases in search complexity. For example, expanding an 8-character password from lowercase ($26^8 \approx 2.08 \times 10^{11}$) to full printable ASCII ($95^8 \approx 6.63 \times 10^{15}$) multiplies the space by roughly 3×10^4 . Conversely, simply increasing a lowercase password from 8 to 16 characters ($26^{16} \approx 4.36 \times 10^{22}$) multiplies the search space by more than 2×10^{11} , highlighting the mathematical advantage of length over alphabet diversity [11].

2.3 Information-Theoretic Entropy

In classical information theory, Claude Shannon defined entropy as the measure of average information content or uncertainty produced by an information source [11]. Assuming characters are chosen independently and uniformly at random from an alphabet of size R , the theoretical character-space entropy estimate E in bits is given by $E = L \times \log_2(R)$.

While this calculation provides an upper-bound baseline under a uniform-character assumption, human users exhibit pronounced cognitive biases: they adhere to phonological constraints, capitalize initial characters, append sequential years or exclamation marks at the tail, and reuse common dictionary words [6, 15]. Consequently, educational heuristic analyzers must penalize predictable patterns rather than presenting character-space formulas as actual entropy of human-generated strings.

2.4 Cryptographic Hashing and Salting

To mitigate risks if an authentication database is breached, passwords must never be stored in plaintext. Cryptographic hash functions provide a deterministic, one-way transformation mapping arbitrary-length inputs into fixed-length digests with pre-image resistance [7] and the Strict Avalanche Criterion [16].

Without salt, identical passwords produce identical hash digests, enabling adversaries to conduct precomputed dictionary lookups using Rainbow Tables [8]. A cryptographic salt is a unique, cryptographically random value (typically 16 bytes or 128 bits) appended to the password prior to hashing: $\text{Stored Digest} = H(\text{Password} || \text{Salt})$. A unique salt forces an attacker to crack each stolen credential individually, neutralizing precomputed multi-target dictionary and rainbow-table attacks [13].

2.5 Fast Hashes vs. Memory-Hard KDFs

General-purpose cryptographic hash functions (such as MD5, SHA-1, and SHA-256) were designed for maximum throughput in digital signatures and file integrity verification [7]. However, this high throughput makes them unsuitable for password storage: massively parallel GPU setups can execute billions of SHA-256 evaluations per second [12]. Current standards—such as NIST SP 800-63B-4 [13]—mandate specialized Key Derivation Functions designed to be deliberately slow and resource-intensive: Argon2id (winner of the Password Hashing Competition) [1], bcrypt (Blowfish-based adaptable cost factor) [10], scrypt (sequential memory-hard algorithm) [9], and PBKDF2 (HMAC iteration) [4].

3. MATERIALS AND METHODS

3.1 Ethical and Privacy Framework

Conducting security research on user credentials carries significant privacy and ethical liabilities [5]. To ensure compliance with academic research standards and user privacy:

- **Local Browser Execution:** The analysis engine executes entirely within the user's local web browser execution context via TypeScript, without transmitting entered text to an application backend.
- **Zero In-Browser Storage:** No persistent state mechanisms (such as localStorage, sessionStorage, cookies, or IndexedDB) are utilized.
- **Purely Synthetic Evaluation Data:** Experimental evaluations are conducted strictly on synthetically generated test strings. No breached passwords, leaked database collections, or live student credentials were collected or stored.

3.2 Heuristic Scoring Formulation

PasswordGuard calculates an educational heuristic score $S \in [0, 100]$ based on additive complexity rewards and subtractive pattern penalties: $S = \text{clamp}[0, 100](S_{\text{length}} + S_{\text{variety}} + B_{\text{diversity}} + B_{\text{passphrase}} - P_{\text{penalties}})$.

- **Length Component ($\$S_{\text{length}}\$, \text{max } 40 \text{ pts})$:** $L < 6$: 4 pts; $6 \leq L \leq 7$: 10 pts; $8 \leq L \leq 11$: 20 pts; $12 \leq L \leq 15$: 32 pts; $L \geq 16$: 40 pts.
- **Character Set Variety ($\$S_{\text{variety}}\$, \text{max } 35 \text{ pts})$:** Lowercase $[a-z]$: +8 pts; Uppercase $[A-Z]$: +8 pts; Digits $[0-9]$: +9 pts; Printable symbols $[\text{'^a-zA-Z0-9}]$: +10 pts.
- **Diversity Bonus ($\$B_{\text{diversity}}\$, \text{max } 15 \text{ pts})$:** 4 pools present: +15 pts; 3 pools: +8 pts; 2 pools: +3 pts.
- **Passphrase Bonus ($\$B_{\text{passphrase}}\$, \text{max } 20 \text{ pts})$:** $L \geq 20$ with ≥ 10 unique chars and no sequential patterns: +20 pts; $L \geq 16$ containing word delimiters: +15 pts.
- **Pattern Penalties ($\$P_{\text{penalties}}\$)$:** Length < 8 : -20 pts ($L < 12$: -8 pts); Repetitive characters: -15 pts; Predictable keyboard/numeric sequences: -15 pts; Common dictionary pattern: -25 pts; Single pool only: -15 pts.

Score tiers are partitioned into: **Weak (0–39)**, **Medium (40–69)**, and **Strong (70–100)**.

3.3 Combinatorial Search Time Estimation

Combinatorial search calculations assume an idealized exhaustive search testing on average half the candidate space: $T = (R^L / 2) / V_{\text{guess}}$, where V_{guess} represents an illustrative assumed guessing rate across three threat profiles: (1) Online Throttled Guessing: $V_1 = 100$ guesses/sec; (2) Offline Desktop Guessing: $V_2 = 10^7$ guesses/sec; (3) High-Performance Cluster Guessing: $V_3 = 10^{11}$ guesses/sec. All displays state that these are illustrative upper-bound calculations [13].

4. SYSTEM DESIGN AND IMPLEMENTATION

4.1 Architecture Overview

PasswordGuard is engineered as a responsive Single-Page Application leveraging React 19, TypeScript, and Vite. The architecture maintains a strict decoupling between heuristic

analysis, cryptographic simulation, and data visualization, operating entirely within client memory.

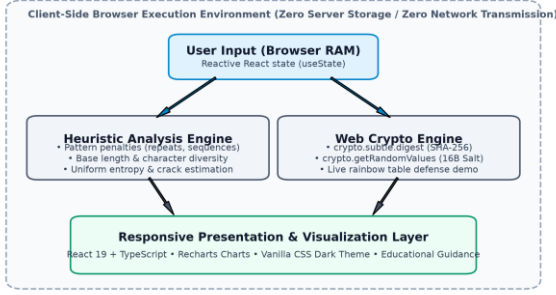


Figure 1. Architectural overview of the PasswordGuard client-side analysis and simulation pipeline.

4.2 Core Modules

1. Analysis Engine ('passwordAnalyzer.ts'): Evaluates candidate strings deterministically, returning pool size R , diversity flags, vulnerability triggers, 0–100 heuristic score, theoretical entropy, and log-space search space $\log_{10}(R^L)$ with overflow protection.

2. Cryptographic Sandbox ('cryptoDemo.ts'): Interacts directly with `window.crypto.subtle.digest('SHA-256')` and 16-byte salts from `window.crypto.getRandomValues()` to provide live, interactive side-by-side comparisons of raw vs. salted hash digests.

3. Visualization Layer: Built with Vanilla CSS dark themes and Recharts data visualization, rendering dynamic score meters, checklists, and experimental distributions.

5. EXPERIMENTAL SETUP

5.1 Objective of the Experiment

The experimental evaluation assesses whether the implemented PasswordGuard heuristic behaves consistently with its design rules across controlled structural categories. It evaluates internal scoring consistency rather than serving as an independent cracking benchmark.

5.2 Synthetic Dataset Construction

To eliminate ethical and privacy risks of handling real credentials [5, 14], a reproducible dataset of 150 synthetic passwords was constructed, evenly divided into five categories ($n = 30$ each):

- **Category A (Short Simple, $L \in [3, 5]$):** 3–5 letter words, occasionally with a digit (e.g., cat, dog, sun, red2).
- **Category B (Common Pattern, $L \in [8, 12]$):** Dictionary roots, capitalized initials, sequential digits and symbols (e.g., password123, admin2024!, qwerty12345).
- **Category C (Medium Complexity, $L \in [11, 15]$):** Mixed alphanumeric strings with symbols adhering to corporate rules (e.g., BlueSky#49, Silver!Fox82).
- **Category D (Long Passphrase, $L \in [25, 36]$):** Multi-word phrases separated by hyphens (e.g., correct-horse-battery-staple, blue-mountains-whisper-softly).
- **Category E (Long Random, $L = 16$):** 16-character pseudo-random strings across all 4 character pools (e.g., 7\$zW#9!kLp&2Qx@m).

5.3 Evaluation Metrics

Each password was processed programmatically, recording character length (L), diversity count ($D \in [1, 4]$), heuristic score ($S \in [0, 100]$), theoretical entropy estimate (E in bits), and categorical tier classification.

6. EXPERIMENTAL RESULTS

The experimental evaluation executed across all 150 synthetic samples without error. The aggregated numerical findings are documented in Table 1.

Table 1. Aggregated PasswordGuard Heuristic Evaluation Results by Category (N = 150)

Cat	Description	n	Mean L	Diversity	Score	Entropy	Tier Dist.
A	Short Simple	30	4.2	1.5	2.0	20.8 b	100% Weak
B	Common Pattern	30	9.6	2.2	10.5	51.2 b	96.7% W, 3.3% M
C	Medium Complex	30	13.0	4.0	80.8	85.4 b	93.3% S, 6.7% M
D	Long Passphrase	30	31.1	2.0	75.8	182.9 b	70.0% S, 30.0% M
E	Long Random	30	16.0	4.0	90.0	105.1 b	100% Strong

6.1 Overall Tier Distribution

Across the entire 150-sample dataset: Weak Tier: 59 passwords (39.3%); Medium Tier: 12 passwords (8.0%); Strong Tier: 79 passwords (52.7%). Figure 2 illustrates this overall distribution.

Overall Password Tier Distribution (N = 150)

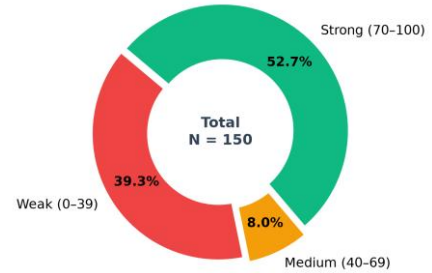


Figure 2. Categorical tier distribution across the entire 150-password synthetic evaluation set.

6.2 Analysis of Category Findings

Category A (Short Simple): Averaged a score of 2.0/100 and theoretical entropy of 20.8 bits. Tiny search spaces ($26^4 \approx 4.5 \times 10^5$) yielded 100% Weak classifications.

Category B (Common Pattern): Demonstrates the penalty engine. Despite containing symbols and numbers (mean length 9.6), dictionary roots and sequences triggered major deductions (-25 common pattern, -15 sequences), resulting in a mean score of 10.5/100 (96.7% Weak).

Category C (Medium Complexity): Traditional corporate composition rules (mean length 13.0, 4 pools) averaged 80.8/100 and 85.4 bits entropy (93.3% Strong).

Category D (Long Passphrase): Multi-word phrases averaging 31.1 characters achieved the highest theoretical character-space

entropy: **182.9 bits**, averaging 75.8/100 (70.0% Strong, 30.0% Medium) despite utilizing only 2 pools.

Category E (Long Random): 16-character pseudo-random strings across all 4 pools achieved the highest score: 90.0/100 and 105.1 bits entropy (100% Strong).

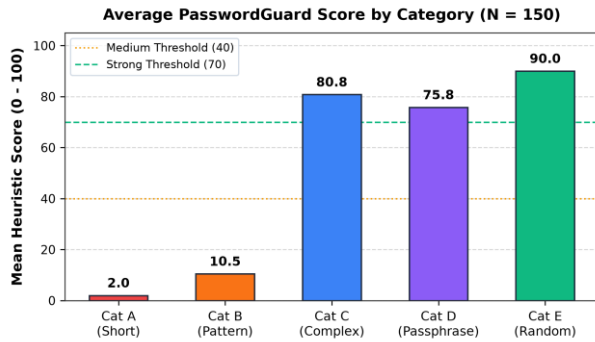


Figure 3. Average PasswordGuard heuristic score across the five synthetic categories (N = 150).

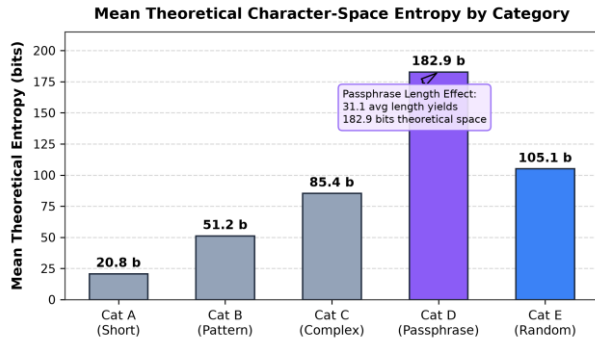


Figure 4. Mean theoretical character-space entropy estimate by category, highlighting passphrase scaling.

7. DISCUSSION

7.1 Methodological Context and Circularity

When interpreting these findings, it is essential to acknowledge the methodological relationship between the heuristic tool and the dataset. Because the same heuristic defines scoring rules and evaluates the synthetic categories, the results reflect the designed model rather than an independent empirical cracking benchmark.

7.2 Limitations of Superficial Complexity

Category B results demonstrate that naive character requirements ($[A-Z]$, $[0-9]$) frequently fail in practice [6]. Attackers employ rule-based and probabilistic grammar attacks targeting predictable substitutions and sequential suffixes [12, 17].

7.3 Passphrases vs. High-Entropy Strings

Category E concentrates 105.1 bits into 16 characters but strains human memory, necessitating password managers [15]. Category D demonstrates that passphrases expand theoretical search spaces (182.9 bits) while maintaining cognitive retention, consistent with NIST SP 800-63B-4 guidance [13].

7.4 Defense in Depth

Password evaluation alone cannot guarantee account defense. Adversaries using phishing or keyloggers bypass complexity

entirely [2]. Robust authentication requires Multi-Factor Authentication (MFA) [13], breached-credential screening [3], and server-side rate-limiting [13].

8. LIMITATIONS

- **Heuristic Scoring Engine:** The 0–100 score is an educational heuristic rather than a formal compliance standard (e.g., FIPS 140-3).
- **Upper-Bound Entropy Assumption:** Theoretical entropy assumes uniform, independent character selection; human cognitive biases result in lower real-world entropy [11, 15].
- **Illustrative Attacker Modeling:** Crack projections assume exhaustive search ($R^L/2$); adversaries prioritize targeted wordlists and mask attacks [12].
- **Educational SHA-256 Boundary:** Web Crypto SHA-256 is used strictly for pedagogical demonstration. Production verifiers require memory-hard KDFs like Argon2id [1, 13].
- **Synthetic Dataset Scope:** 150 synthetic samples isolate structural variables safely but do not reflect the statistical distribution of general human populations [5, 14].

9. CONCLUSION

This project developed and evaluated **PasswordGuard**, an educational client-side web application exploring password security and authentication defense. By executing locally in browser memory without server-side persistence, the tool aims to balance technical clarity, educational value, and privacy-oriented design. Evaluation on 150 synthetic passwords demonstrates how the heuristic behaves: length mathematically scales combinatorial search spaces (R^L), enabling memorable passphrases to achieve high theoretical entropy (182.9 bits). Ultimately, password resilience must be paired with defense in depth: password managers, slow memory-hard hashing (Argon2id), rate limiting, and Multi-Factor Authentication (MFA) [13].

10. ACKNOWLEDGMENTS

The authors thank the course instructors of English Writing and Presentation Skills, Faculty of Information Technology, Vietnam National University, for valuable academic guidance and feedback throughout this project.

11. REFERENCES

- [1] Biryukov, A., Dinu, D., and Khovratovich, D. 2016. Argon2: New generation of memory-hard functions for password hashing and other applications. In Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P '16). IEEE, 292–302. DOI: <https://doi.org/10.1109/EuroSP.2016.31>.
- [2] Bonneau, J., Herley, C., van Oorschot, P. C., and Stajano, F. 2012. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P '12). IEEE, 553–567. DOI: <https://doi.org/10.1109/SP.2012.44>.
- [3] Hunt, T. 2018. I've Just Launched Pwned Passwords V2 with Half a Billion Passwords. Troy Hunt Blog (Feb. 22, 2018). Retrieved from <https://www.troyhunt.com/ive-just-launched-pwned-passwords-version-2/>
- [4] Kaliski, B. 2000. PKCS #5: Password-Based Cryptography Specification Version 2.0. RFC 2898. Internet Engineering Task Force. DOI: <https://doi.org/10.17487/RFC2898>.

- [5] Kenneally, E. and Dittrich, D. 2012. The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research. Technical Report. U.S. Department of Homeland Security.
- [6] Komanduri, S., Shay, R., Kelley, P. G., Mazurek, M. L., Bauer, L., Christin, N., Cranor, L. F., and Egelman, S. 2011. Of passwords and people: Measuring the effect of password-composition policies. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)*. ACM, 2595–2604. DOI: <https://doi.org/10.1145/1978942.1979321>.
- [7] National Institute of Standards and Technology. 2015. Secure Hash Standard (SHS). Federal Information Processing Standards Publication (FIPS PUB) 180-4. U.S. Department of Commerce. DOI: <https://doi.org/10.6028/NIST.FIPS.180-4>.
- [8] Oechslin, P. 2003. Making a faster cryptanalytic time-memory trade-off. In *Advances in Cryptology — CRYPTO 2003. Lecture Notes in Computer Science*, vol. 2729. Springer, 617–630. DOI: https://doi.org/10.1007/978-3-540-45146-4_36.
- [9] Percival, C. 2009. Stronger key derivation via sequential memory-hard functions. In *Proceedings of BSDCan 2009: The Technical BSD Conference (Ottawa, Canada, May 2009)*. (Standardized in IETF RFC 7914, 2016).
- [10] Provos, N. and Mazières, D. 1999. A future-adaptable password scheme. In *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. USENIX Association, 81–91.
- [11] Shannon, C. E. 1948. A mathematical theory of communication. *Bell System Technical Journal* 27, 3 (1948), 379–423. DOI: <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>.
- [12] Steube, J. 2020. Hashcat: Advanced password recovery utility. Documentation and benchmark architecture. <https://hashcat.net/hashcat/>
- [13] Temoshok, D., Fenton, J. L., Choong, Y.-Y., Lefkovitz, N., Regenscheid, A., Galluzzo, R., and Richer, J. P. 2025. Digital Identity Guidelines: Authentication and Authenticator Management. NIST Special Publication 800-63B-4. National Institute of Standards and Technology. DOI: <https://doi.org/10.6028/NIST.SP.800-63B-4>.
- [14] Thomas, K., Li, F., Zand, A., Barrett, J., Ranieri, G., Invernizzi, L., Markov, Y., Comanescu, O., Eranti, V., Moscicki, A., Margolis, D., Paxson, V., and Bursztein, E. 2017. Data breaches, phishing, or malware? Understanding the risks of stolen credentials. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 1421–1434. DOI: <https://doi.org/10.1145/3133956.3134067>.
- [15] Ur, B., Segreti, S. M., Bauer, L., Christin, N., Cranor, L. F., Komanduri, S., Meckling, R., Wash, R., and Mazurek, M. L. 2015. Measuring real-world accuracies and biases in modeling password guessability. In *Proceedings of the 24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, 963–981.
- [16] Webster, F. L. and Tavares, S. E. 1985. On the design of S-boxes. In *Advances in Cryptology — CRYPTO '85. Lecture Notes in Computer Science*, vol. 218. Springer, 523–534. DOI: https://doi.org/10.1007/3-540-39799-X_41.
- [17] Weir, M., Aggarwal, S., de Medeiros, B., and Glodek, B. 2009. Password cracking using probabilistic context-free grammars. In *Proceedings of the 2009 IEEE Symposium on Security and Privacy (S&P '09)*. IEEE, 391–405. DOI: <https://doi.org/10.1109/SP.2009.8>.